Fake News Detector Project

This project implements a simple fake news detector with a Flask web interface. It uses a transformer-based model for text classification.

Project Structure

```
fake_news_detector/
├── README.md              # This file
├── requirements.txt       # Python dependencies
├── saved_models/          # Directory to store trained models (e.g., your Colab model)
│   └── fake_news_model/   # Example: placeholder for your actual model files
├── src/
│   ├── main.py            # Flask application entry point
│   ├── model/
│   │   └── model.py       # Fake news detection model logic (loading, prediction)
│   ├── static/
│   │   └── style.css      # CSS for the web interface
│   └── templates/
│       └── index.html     # HTML template for the web interface
├── notebooks/             # For Jupyter notebooks (e.g., EDA, model experimentation)
└── tests/                 # For unit and integration tests
```

Setup and Running Locally (e.g., in VS Code)

1. **Clone/Download the Project:** Ensure you have all the files from the provided zip archive.

2. **Create a Virtual Environment (Recommended):** Open your terminal or VS Code terminal in the `fake_news_detector` project root directory. `bash python3 -m venv venv source venv/bin/activate # On Windows: venv\Scripts\activate`

3. **Install Dependencies:** `bash pip install -r requirements.txt`

4. **Using Your Custom Model (Trained in Colab):**

    - **Save your model in Colab:** After training/fine-tuning your model in Google Colab, save it using the `save_pretrained` method: `python # In your Colab notebook model.save_pretrained("./my_actual_fake_news_model") tokenizer.save_pretrained("./my_actual_fake_news_model")`

- **Download the model files:** Download the entire `my_actual_fake_news_model` directory (which will contain `config.json`, `pytorch_model.bin` or `model.safetensors`, `tokenizer_config.json`, `vocab.txt`, etc.) from Colab to your local machine.
- **Place the model locally:**
  - Create the `saved_models` directory if it doesn't exist in your project root: `fake_news_detector/saved_models/`
  - Place your downloaded `my_actual_fake_news_model` directory inside `saved_models`. So the path would be `fake_news_detector/saved_models/my_actual_fake_news_model/`.
- **Update Model Path in Code:** Open `src/model/model.py`. Modify the `MODEL_PATH` variable to point to your model directory: `python # In src/model/model.py # ... # MODEL_PATH = os.path.join(os.path.dirname(__file__), "..", "..", "saved_models", "my_actual_fake_news_model") # Or, if you want to load it by its Hugging Face Hub name (if uploaded) or the default path: # DEFAULT_MODEL_NAME = "your-username/your-model-name-on-hub" # If uploaded to Hub # detector = FakeNewsDetector(model_name_or_path="saved_models/my_actual_fake_news_model") # To load from specific path` The current `model.py` tries to load from `saved_models/fake_news_model` by default if it exists. You can rename your downloaded model folder to `fake_news_model` and place it in `saved_models` for the easiest setup, or adjust the `MODEL_PATH` or `DEFAULT_MODEL_NAME` in `src/model/model.py` as shown in its comments.

5. **Run the Flask Application:** Navigate to the project root directory (`fake_news_detector`) in your terminal (ensure your virtual environment is active). Run the application using: `bash python src/main.py` The application should start, and you'll see output indicating it's running (usually on `http://127.0.0.1:5000/`).

6. **Access in Browser:** Open your web browser and go to `http://127.0.0.1:5000/`.

Notes

- The current `src/model/model.py` uses `distilbert-base-uncased-finetuned-sst-2-english` (a sentiment analysis model) as a placeholder if no local model is found or specified. This is for demonstration purposes. **You MUST replace this with your actual fake news detection model for meaningful results.** The labels (e.g., "REAL", "FAKE") and their interpretation will depend on how your specific model was trained and its configuration (`config.json`).

- The `model.py` script includes an example `if __name__ == "__main__":` block that you can use to test model loading and prediction directly from the command line before running the Flask app.
- For production, consider using a production-grade WSGI server like Gunicorn instead of Flask's built-in development server. `bash # Example: running with Gunicorn (from project root, venv active) # gunicorn --workers 4 --bind 0.0.0.0:5000 src.main:app`

This `README.md` provides a guide to setting up and running the project.